PLAYWRIGHT CLASS NOTES

Handling Date Pickers

Beginner-to-Professional Guide | DOM Structures | Real-World Scenarios | Code Walkthroughs

How to use this guide: Start with the DOM diagrams and Demo 1. Run each example yourself. When the page behaves differently from the example, inspect the live DOM and adapt the selector instead of copying a locator blindly.

Learning outcomes

- Understand the difference between a normal text date field, a native HTML date input, and a custom calendar widget.
- Read the DOM structure and identify the input, month/year heading, navigation controls, and day cells.
- Enter a date directly when the application allows it.
- Open a calendar, navigate to the required month/year, and select the right day.
- Understand the two date-selection approaches from class: loop through date elements or use a parameterized XPath.
- Build safer tests with assertions, bounded loops, clear locators, and realistic test cases.

1. What is a date picker?

A date picker is a user-interface control that lets someone choose a date. It may display a calendar popup with the current month, year, day cells, and controls for moving to previous or next months. Websites do not all use the same date picker because it may come from a browser, a UI library, or application-specific code.

Type	What you see	Typical automation approach
Editable text input	A field such as 10/29/2026; typing is allowed.	Use locator.fill() and verify the resulting value.
Native date input	Browser-controlled date UI, often shown as month/day/year or a localized layout.	Use fill() with the input value format YYYY-MM-DD, then verify the value.
Custom calendar widget	A calendar popup with month/year labels and selectable day cells.	Open it, navigate to the target month/year, select the day, and assert the result.
Restricted date picker	Typing may be blocked; past dates or certain dates may be disabled.	Interact with the widget and verify business rules such as disabled dates.

Important distinction: The class file uses a jQuery UI-style custom date picker on https://testautomationpractice.blogspot.com/. The selectors in these notes match that style. Another website may use different classes, labels, or DOM structure.

2. Read the DOM before writing a locator

DOM stands for Document Object Model. In practical terms, it is the browser representation of the HTML elements. A Playwright locator must match an element that actually exists in the current DOM.

2.1 Input field before the calendar opens

```
<label for="datepicker">Date:</label>
<input type="text" id="datepicker">
```

Selector guide:

- #datepicker is a CSS ID selector. The # symbol means “the element whose id is datepicker”.

- page.locator("#datepicker") locates the input.
- fill("10/29/2026") sets the input value when the app permits manual text entry.

2.2 Simplified calendar popup DOM

```
<div id="ui-datepicker-div">
 <div class="ui-datepicker-header">
  <a title="Prev">Previous month</a>
  <div class="ui-datepicker-title">
   <span class="ui-datepicker-month">September</span>
   <span class="ui-datepicker-year">2025</span>
  </div>
  <a title="Next">Next month</a>
 </div>
 <table class="ui-datepicker-calendar">
  <tbody>
   <tr>
    <td data-handler="selectDay" data-month="8" data-year="2025">
     <a class="ui-state-default">13</a>
    </td>
   </tr>
  </tbody>
 </table>
</div>
```

This is a simplified teaching diagram, not a promise that every live calendar has exactly this markup. In the jQuery UI widget, the month value in data-month is zero-based: January is 0, February is 1, and September is 8.

Selector in class	Meaning
#datepicker	Input field that opens the calendar.
.ui-datepicker-month	Current month text in the popup.
.ui-datepicker-year	Current year text in the popup.
[title="Prev"]	Element whose title attribute equals Prev.
[title="Next"]	Element whose title attribute equals Next.
.ui-datepicker-calendar	Table containing calendar day cells.
td[data-handler="selectDay"]	A calendar table cell set up for day selection.

3. Demo 1 - Enter a date manually

Scenario from class: open the practice page and type 29 October 2026 directly into the date field. This works only if the field accepts typed input and the date format matches the application.

```
import { test, expect } from '@playwright/test';

test('Date Picker Demo 1 - enter date manually', async ({ page }) => {
 await page.goto('https://testautomationpractice.blogspot.com/');

 const dateInput = page.locator('#datepicker');
 await dateInput.fill('10/29/2026');

 // Verify that the value is in the input after filling it.
 await expect(dateInput).toHaveValue('10/29/2026');
});
```

Line-by-line explanation

1. import { test, expect } imports Playwright Test functions. test() defines the test; expect() checks the result.

2. async ({ page }) receives the page fixture. page represents the browser tab used by this test.
3. page.goto(url) opens the practice website. await is needed because navigation is asynchronous.
4. page.locator("#datepicker") finds the input with id="datepicker".
5. fill("10/29/2026") enters the date text. It does not prove that the application accepted it, so the assertion matters.
6. toHaveValue(...) verifies the input field value. This assertion also waits for the expected value until its timeout.

Do not confuse date formats: The class example uses MM/DD/YYYY, such as 10/29/2026. A native <input type="date"> normally uses a value like 2026-10-29. Always inspect the input type and application configuration first.

3.1 What if direct typing is blocked?

Some controls are read-only or validate input only through the calendar UI. In that case, do not keep trying fill() repeatedly. Open the calendar and interact with its month/year navigation and day cells.

4. Native HTML date input versus custom calendar

```
<!-- Native date input -->
<label for="birthDate">Birth date</label>
<input type="date" id="birthDate">

// Example for a native <input type="date">
const birthDate = page.locator('#birthDate');
await birthDate.fill('1998-04-15');
await expect(birthDate).toHaveValue('1998-04-15');
```

For native date inputs, the DOM value is commonly ISO format YYYY-MM-DD regardless of how the browser visually formats the control. Do not assume the visible date text and input value always use the same format.

5. Demo 2 - Navigate to the target month and year

The class target is 13 September 2025. The script opens the calendar, reads the currently displayed month and year, and compares them with the target. Once the calendar shows the target month and year, it selects day 13.

5.1 The class approach: click Previous until the month matches

```
const targetYear = '2025';
const targetMonth = 'September';
const targetDate = '13';

await page.locator('#datepicker').click();

while (true) {
 const currentYear = (await page.locator('.ui-datepicker-year').textContent())?.trim();
 const currentMonth = (await page.locator('.ui-datepicker-month').textContent())?.trim();

 if (currentYear === targetYear && currentMonth === targetMonth) {
  break;
 }

 await page.locator('[title="Prev"]').click();
}
```

How to understand it:

- targetYear, targetMonth, and targetDate store the date your test wants to choose.
- Clicking #datepicker opens the custom calendar popup.

- textContent() reads the text displayed by a locator. trim() removes extra spaces around the text.
- The if condition checks whether both year and month are correct. break exits the loop once they match.
- The last line clicks Previous, which is suitable only if the target is earlier than the currently displayed month.

A critical limitation in the original class code: while (true) with only a Previous click can loop forever if the target month is in the future, if the selector is wrong, or if the calendar cannot move further back. Use a bounded loop and support both Previous and Next.

5.2 Safer version: navigate backwards or forwards with a limit

```
import { test, expect } from '@playwright/test';

test('Select a date by calendar navigation', async ({ page }) => {
 await page.goto('https://testautomationpractice.blogspot.com/');

 const targetYear = 2025;
 const targetMonth = 'September';
 const targetDate = '13';
 const months = [
  'January', 'February', 'March', 'April', 'May', 'June',
  'July', 'August', 'September', 'October', 'November', 'December'
 ];

 await page.locator('#datepicker').click();

 const targetMonthIndex = months.indexOf(targetMonth);
 if (targetMonthIndex === -1) {
  throw new Error(`Unknown target month: ${targetMonth}`);
 }
 const targetIndex = targetYear * 12 + targetMonthIndex;
 let reachedTargetMonth = false;

 for (let attempt = 0; attempt < 120; attempt++) {
  const yearText = (await page.locator('.ui-datepicker-year').textContent())?.trim();
  const monthText = (await page.locator('.ui-datepicker-month').textContent())?.trim();
  const visibleYear = Number(yearText);
  const visibleMonthIndex = months.indexOf(monthText ?? '');

  if (!Number.isFinite(visibleYear) || visibleMonthIndex === -1) {
   throw new Error(`Could not read calendar month/year: ${monthText} ${yearText}`);
  }

  const visibleIndex = visibleYear * 12 + visibleMonthIndex;
  if (visibleIndex === targetIndex) {
   reachedTargetMonth = true;
   break;
  }

  if (visibleIndex < targetIndex) {
   await page.locator('[title="Next"]').click();
  } else {
   await page.locator('[title="Prev"]').click();
  }
 }

 if (!reachedTargetMonth) {
  throw new Error('The calendar did not reach the target month within 120 attempts.');
 }

 await expect(page.locator('.ui-datepicker-month')).toHaveText(targetMonth);
 await expect(page.locator('.ui-datepicker-year')).toHaveText(String(targetYear));
```

```
 // Use a selector tied to the target year, month, and day to avoid duplicate day numbers.
 const dayCell = page.locator(
  `//td[@data-handler='selectDay' and @data-month='${targetMonthIndex}' and @data-year='${targetYear}']/a[normalize-space()='${targetDate}']`
 );
 await expect(dayCell).toBeVisible();
 await dayCell.click();


 await expect(page.locator('#datepicker')).toHaveValue('09/13/2025');
});
```

This is a teaching example for the jQuery UI-style calendar from the class. If the website uses a different date format or markup, adjust the final expected value and selectors after inspecting the live DOM.

6. Approach A - Find the day by looping through date elements

The original class captures a collection of calendar date elements and checks each element's text until it matches the requested day. This is useful for understanding iteration, but the elements should be scoped to the correct month/year to avoid clicking a duplicate number from an adjacent month.

```
const targetYear = 2025;
const targetMonth = 'September';
const targetDate = '13';
const months = [
 'January', 'February', 'March', 'April', 'May', 'June',
 'July', 'August', 'September', 'October', 'November', 'December'
];
const targetMonthIndex = months.indexOf(targetMonth); // September = 8


const dates = page.locator(
 `.ui-datepicker-calendar td[data-handler="selectDay"][data-month="${targetMonthIndex}"][data-year="${targetYear}"] a`
);
const count = await dates.count();
let dateSelected = false;


for (let i = 0; i < count; i++) {
 const dayLink = dates.nth(i);
 const visibleDay = (await dayLink.textContent())?.trim();


 if (visibleDay === targetDate) {
  await dayLink.click();
  dateSelected = true;
  break;
 }
}


if (!dateSelected) {
 throw new Error(`Day ${targetDate} was not found in ${targetMonth} ${targetYear}`);
}
```

Understand each part

- locator(...) creates a locator for the day links in the target calendar month and year.
- count() tells you how many matching elements exist.
- nth(i) refers to the element at zero-based index i: the first item is nth(0).
- textContent() reads the day number. trim() removes whitespace.
- The if statement compares the displayed day number with the target date. When it matches, the script clicks it and exits the loop.
- dateSelected records whether a matching element was clicked. The explicit error is more helpful than silently passing when the date was not selected.

Modern locator best practice: Prefer locator() plus count()/nth() over the older page.$$() ElementHandle-array approach for most new Playwright tests. A locator is re-evaluated against the current DOM, which is helpful when a calendar re-renders.

7. Approach B - Select the day with a parameterized XPath

The second technique in the supplied notes builds an XPath by inserting the date variable into a template literal. A template literal uses backticks (`) and allows JavaScript values inside ${...}.

```
const date = '13';
await page.locator(`//a[@class='ui-state-default'][text()='${date}']`).click();
```

What is happening?

- The backticks make this a template literal rather than a normal single-quoted string.
- ${date} is replaced with the value stored in the date variable. In this example, the final XPath looks for an anchor with the text 13.
- XPath uses // to search descendant elements, @class to check an attribute, and text() to look at the text node.

Why the simple XPath can be fragile: A calendar can display trailing days from the previous month and leading days from the next month. The number 13 could appear more than once in the visible grid. The exact class could also change. Scope the day to the target calendar cell using data-month and data-year when those attributes exist.

```
// More precise XPath for 13 September 2025 (September index = 8)
const day = page.locator(
  `//td[@data-handler='selectDay' and @data-month='8' and @data-year='2025']/a[normalize-space()='13']`
);
await expect(day).toBeVisible();
await day.click();
```

8. Real-world automation scenarios

Scenario 1 - Book a doctor appointment

Business goal: choose a particular appointment date and make sure the booking form holds the intended date.

```
const appointmentDate = page.getByLabel('Appointment date');
await appointmentDate.fill('11/05/2026');
await expect(appointmentDate).toHaveValue('11/05/2026');
```

If manual entry is disabled, use the calendar-navigation pattern, then assert the chosen date. The selector and format depend on the actual booking page.

Scenario 2 - Hotel check-in and check-out

```
const checkIn = page.getByLabel('Check-in');
const checkOut = page.getByLabel('Check-out');

await checkIn.fill('11/12/2026');
await checkOut.fill('11/15/2026');

await expect(checkIn).toHaveValue('11/12/2026');
await expect(checkOut).toHaveValue('11/15/2026');
```

Test the business rule too: check-out must be after check-in. A good test checks that the app rejects invalid date ranges or disables dates that violate the rule.

Scenario 3 - Flight search

Flight search forms may use one-way or return dates. Verify that the selected departure date is preserved, that the return date is not earlier than departure, and that the calendar does not allow invalid dates if this is an application requirement.

Scenario 4 - Report filter

An admin dashboard may have Start date and End date fields. Choose the reporting range, click Apply, and assert the displayed date range or resulting report heading. Checking the input alone may not prove that the report filter was applied.

Scenario 5 - Date of birth

Date-of-birth controls may prevent future dates and sometimes restrict the year range. Test both a valid date and an invalid future date. Use synthetic test data, not real customer data.

9. Disabled dates and date-range validation

A calendar may show dates that cannot be selected. In a jQuery UI date picker, a disabled date may use classes such as ui-datepicker-unselectable rather than a clickable day link. The exact markup depends on the widget.

```
// General test idea - first inspect the live DOM and use a stable selector.
const disabledDay = page.locator('.ui-datepicker-unselectable');
await expect(disabledDay.first()).toBeVisible();

// If the site exposes a disabled state/accessibility attribute, assert that state
// using the locator that matches the actual markup.
```

Do not assume the selector above is valid for every date picker. If the date is disabled through CSS or custom JavaScript, inspect the actual element and use the most reliable accessible state or application behavior available.

10. Common errors and how to fix them

Problem	Likely cause	What to do
fill() does not set the date	Input is read-only, custom-controlled, or expects another format.	Inspect type/readonly attributes; use calendar interaction or the correct format.
Month never matches	Wrong selector, unexpected month text, or navigation only goes one direction.	Log month/year, inspect the DOM, support Next and Prev, and cap the loop.
The wrong day gets selected	Duplicate day numbers from adjacent months.	Scope by data-month and data-year; do not target text alone.
Timeout on click	Locator is incorrect, date is disabled, overlay is closed, or element is not actionable.	Inspect the call log and live DOM; assert visibility and enabled state where applicable.
Assertion expects the wrong value	Display format differs from field value format.	Inspect the input value and widget configuration; use the expected format actually used by the app.
Test passes only with waitForTimeout	Timing or synchronization is unreliable.	Replace the fixed sleep with a web-first assertion on the visible month/year or final input value.

11. Best practices for reliable date-picker tests

- Inspect the DOM before writing selectors. Confirm the input type, calendar popup structure, month/year labels, navigation controls, and day cell attributes.
- Prefer accessible locators such as getByLabel() or getByRole() when they uniquely identify the control. For calendar internals with no accessible names, use stable CSS attributes or a carefully scoped XPath.
- Prefer locator() APIs over old ElementHandle collections for new tests. Locators re-resolve against the DOM and support Playwright auto-waiting.
- Assert the final input value or the selected state. A click action by itself does not prove that the intended date was accepted.
- Avoid waitForTimeout(4000) as a normal synchronization strategy. Assert the month/year label or the final input value instead.
- Never use unbounded while(true) in a date navigation loop unless there is a guaranteed break condition and a protective limit. A maximum-attempt loop is safer.
- Use target month and year when locating the day, because day numbers repeat across months and adjacent-month cells may appear in the grid.
- Keep date values in variables so a test can be reused. Prefer clear names such as targetYear, targetMonth, and targetDate.
- Use exact, known test data and consider month length, leap years, disabled dates, locale, timezone, and date format.
- Keep tests independent. Each test should navigate to the page and establish its own starting state.

Reusable workflow: 1) Locate and open the input. 2) Read the visible month/year. 3) Click Next or Previous until the target month appears, with a maximum attempt count. 4) Locate the day inside the target year and month. 5) Select it and assert the input value.

12. Complete reusable example for the class practice page

This consolidates the safer pattern: open the calendar, navigate in either direction, select a precisely scoped date, and verify the input value. It assumes the practice page continues using the jQuery UI-style DOM shown earlier.

```
import { test, expect } from '@playwright/test';

test('Choose 13 September 2025 from the calendar', async ({ page }) => {
 await page.goto('https://testautomationpractice.blogspot.com/');

 const dateInput = page.locator('#datepicker');
 const targetYear = 2025;
 const targetMonth = 'September';
 const targetDate = '13';
 const monthNames = [
  'January', 'February', 'March', 'April', 'May', 'June',
  'July', 'August', 'September', 'October', 'November', 'December'
 ];

 await dateInput.click();

 const targetMonthNumber = monthNames.indexOf(targetMonth);
 if (targetMonthNumber < 0) throw new Error('Invalid target month');
 const targetSerial = targetYear * 12 + targetMonthNumber;
 let foundMonth = false;

 for (let step = 0; step < 120; step++) {
  const year = Number((await page.locator('.ui-datepicker-year').innerText()).trim());
  const month = (await page.locator('.ui-datepicker-month').innerText()).trim();
  const monthNumber = monthNames.indexOf(month);
  if (monthNumber < 0 || !Number.isFinite(year)) {
   throw new Error(`Unable to read calendar header: ${month} ${year}`);
  }

  const visibleSerial = year * 12 + monthNumber;
  if (visibleSerial === targetSerial) {
   foundMonth = true;
   break;
  }
  await page.locator(visibleSerial < targetSerial ? '[title="Next"]' : '[title="Prev"]').click();
 }

 if (!foundMonth) throw new Error('Target month was not reached after 120 calendar moves');
 await expect(page.locator('.ui-datepicker-month')).toHaveText(targetMonth);
 await expect(page.locator('.ui-datepicker-year')).toHaveText(String(targetYear));

 const day = page.locator(
  `//td[@data-handler='selectDay' and @data-month='${targetMonthNumber}' and @data-year='${targetYear}']/a[normalize-space()='${targetDate}']`
 );
 await expect(day).toBeVisible();
 await day.click();

 await expect(dateInput).toHaveValue('09/13/2025');
});
```

Before running this complete example: Confirm the website still uses #datepicker, .ui-datepicker-month, .ui-datepicker-year, title="Prev"/title="Next", and the data-month/data-year calendar cells. If the DOM changes, adjust the selectors to what the browser actually renders.

13. Beginner practice exercises

Exercise 1 - Manual date

Fill 10/29/2026 in #datepicker and assert the input value.

Exercise 2 - Pick a past date

Use the popup to navigate to September 2025 and choose the 13th.

Exercise 3 - Pick a future date

Change the target to a month after the currently displayed month. Confirm your navigation can click Next, not only Prev.

Exercise 4 - Avoid duplicate days

Inspect a month grid and find whether days from adjacent months are displayed. Scope your target locator by year and month.

Exercise 5 - Replace a fixed wait

Remove waitForTimeout(4000) and assert the final date value with expect(...).toHaveValue(...).

Exercise 6 - Negative test

Try to select a date that the application marks disabled. Assert that it cannot be selected or that the application shows a validation message.

Exercise 7 - Reusable helper

Move month navigation into a helper function such as selectCalendarDate(page, year, month, day). Keep a maximum number of attempts and useful errors.

14. Quick reference

Task	Example
Open the practice page	await page.goto('https://testautomationpractice.blogspot.com/');
Locate date input	const dateInput = page.locator('#datepicker');
Type a date	await dateInput.fill('10/29/2026');
Verify text value	await expect(dateInput).toHaveValue('10/29/2026');
Read displayed month	await page.locator('.ui-datepicker-month').innerText();
Click previous month	await page.locator('[title="Prev"]').click();
Click next month	await page.locator('[title="Next"]').click();
Find date by month/year	td[data-handler="selectDay"][data-month="8"][data-year="2025"] a
Assert month	await expect(page.locator('.ui-datepicker-month')).toHaveText('September');

15. Further reading

The examples and class terminology in this guide are based primarily on the supplied DatePicker-flow.txt. The following official references support the general Playwright and jQuery UI concepts:

Playwright input actions: https://playwright.dev/docs/input | Playwright locators: https://playwright.dev/docs/locators | jQuery UI Datepicker API and widget structure: https://api.jqueryui.com/datepicker/